Guía de estudio – FastExpress

Oct 14, 2026 · @GAMS

1. Qué es FastExpress y sus 4 reglas

FastExpress es una página web que le dice a un cliente **cuánto cuesta enviar un paquete**. El usuario escribe tres datos (peso, tipo de cliente y tipo de envío) y el sistema calcula el precio con cuatro reglas de negocio.

El proyecto tiene dos partes que se califican por igual: **construir** el módulo y **demostrar con pruebas** que calcula bien. Por eso la mayor parte del trabajo es de calidad de software.

Regla	Qué dice	En palabras simples
BR-01 Tarifa base	$5.00 por cada orden	Todo envío cuesta al menos $5
BR-02 Peso	0.01 a 5.00 kg: $2.00 por kg. 5.01 a 20.00 kg: $4.00 por kg. Más de 20 kg: se rechaza	Paquetes livianos pagan menos por kilo; más de 20 kg no se aceptan
BR-03 Tipo de cliente	Nuevo 0 %, Frecuente 10 %, VIP 20 % de descuento	Los clientes fieles pagan menos
BR-04 Tipo de envío	Estándar 0 %, Express +25 % de recargo	El envío rápido cuesta más

La fórmula, paso a paso

1. **Subtotal** = $5.00 + peso × tarifa por kg.
2. **Recargo** = subtotal × 25 % si es Express (si no, $0).
3. **Descuento** = (subtotal + recargo) × porcentaje del cliente.
4. **Total** = subtotal + recargo − descuento.

Cada monto se redondea a centavos (0.005 sube a 0.01).

Tres ejemplos para practicar

Caso	Cálculo	Total
3 kg, Nuevo, Estándar	5 + 3 × 2 = 11; sin recargo ni descuento	**$11.00**
10 kg, Frecuente, Express	5 + 10 × 4 = 45; recargo 45 × 0.25 = 11.25; descuento (45 + 11.25) × 0.10 = 5.63; 45 + 11.25 − 5.63	**$50.62**
20 kg, Nuevo, Express (el más caro posible)	5 + 20 × 4 = 85; recargo 21.25; sin descuento	**$106.25**

El envío más barato posible es 0.01 kg, VIP, Estándar: 5 + 0.02 = 5.02, menos 20 % = **$4.02**.

Supuestos que decidió el equipo

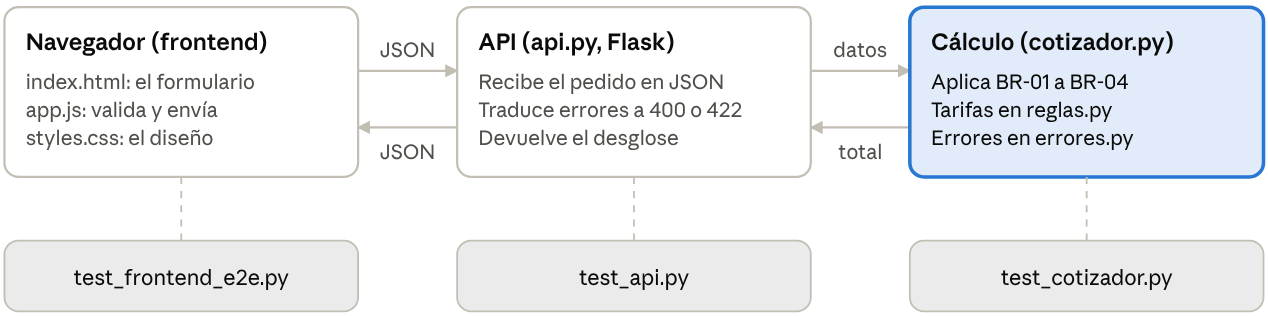
El enunciado no aclaraba algunos puntos, así que el equipo los decidió y los escribió en el informe (sección 1.2):

- Todo el peso se cobra con la tarifa de su tramo: 10 kg = 10 × $4, no "5 kg a $2 y 5 kg a $4".
- El descuento se aplica después del recargo. Como ambos son porcentajes, el total sale igual en cualquier orden.
- "Monto" se entiende como el total cotizado, porque ninguna regla define un monto de entrada.
- Un peso de 0, negativo o con letras es un error de entrada.

2. Cómo funciona por dentro

El programa tiene tres partes que se pasan el trabajo como una cadena: la página pide, la API recibe y el cálculo resuelve. Debajo de cada parte está el archivo de pruebas que la vigila.

Cada parte tiene su tarea y su propia prueba



arquitectura de FastExpress · 3 partes y sus pruebas

1. **Frontend (lo que ve el usuario).** El usuario llena el formulario y pulsa Cotizar. `app.js` revisa primero lo básico (que el peso sea un número entre 0.01 y 20 y que los dos tipos estén elegidos). Si algo falta, muestra el mensaje en rojo y no envía nada.
2. **API (el mensajero).** Si todo está bien, `app.js` envía los datos en formato JSON a `POST /api/cotizaciones`. `api.py` los recibe y llama a la función de cálculo.
3. **Cálculo (el cerebro).** `calcular_cotizacion()` en `cotizador.py` valida otra vez, aplica las 4 reglas y devuelve el desglose. Si hay un error, lanza una excepción con un código, como `LIMITE_EXCEDIDO`.
4. **Respuesta.** La API traduce el resultado a un código HTTP y el frontend pinta la tabla o el mensaje.

Código HTTP	Significa	Ejemplo
200	Todo bien, aquí está la cotización	10 kg, VIP, Express → $45.00
400	Dato mal escrito o faltante	peso "abc" → `PESO_INVALIDO`
422	Dato válido pero fuera del límite	25 kg → `LIMITE_EXCEDIDO`

¿Por qué se valida dos veces? El frontend valida para dar una respuesta rápida al usuario. El backend valida porque cualquiera podría llamar a la API sin pasar por la página; la regla real vive en el servidor.

¿Por qué se usa `Decimal` y no números normales? Con números normales, 0.1 + 0.2 da 0.30000000000000004. En dinero eso produce centavos erróneos, así que todo el cálculo usa `Decimal` con redondeo comercial.

3. Pruebas de caja negra

En caja negra se prueba **sin mirar el código**: solo se usan las reglas para decidir qué datos meter y qué resultado esperar. Es como probar una calculadora sin abrirla. Se usaron tres técnicas y salieron 34 casos automatizados y 20 casos manuales.

3.1 Particiones de equivalencia (EP)

Idea: si el programa trata igual a todos los valores de un grupo, basta probar uno de cada grupo. Así no hay que probar 2, 3, 4, 5… kg uno por uno.

Grupo (partición)	Rango	¿Válido?	Representante usado	Resultado esperado
Tramo 1	0.01 a 5.00 kg	Sí	2.50 kg	$10.00
Tramo 2	más de 5.00 hasta 20.00 kg	Sí	12.00 kg	$53.00
Bajo el mínimo	menos de 0.01 (0 o negativo)	No	−5 kg	Error `PESO_INVALIDO`
Sobre el límite	más de 20.00 kg	No	25 kg	Error `LIMITE_EXCEDIDO`
No numérico	texto, vacío	No	"abc"	Error `PESO_INVALIDO`

3.2 Análisis de valores límite (BVA)

Idea: los errores de programación se esconden en los bordes. Un programador puede escribir `<` donde iba `<=`. Por eso se prueba **cada borde, un paso antes y un paso después** (BVA de tres valores, paso de 0.01 kg).

Borde	Un paso antes	El borde	Un paso después
Mínimo 0.01 kg	0.00 → error	0.01 → $5.02	0.02 → $5.04
Cambio de tarifa 5.00 kg	4.99 → $14.98	5.00 → $15.00	5.01 → $25.04
Máximo 20.00 kg	19.99 → $84.96	20.00 → $85.00	20.01 → error

Fíjate en el salto de 5.00 a 5.01 kg: el precio sube de $15.00 a $25.04 porque cambia la tarifa por kilo. Ese salto es justo lo que el BVA busca confirmar.

Monto: el monto no es un dato que se escribe, es el total que sale. Por eso sus límites se buscan con las entradas que lo producen: el mínimo posible es $4.02 (0.01 kg, VIP, Estándar) y el máximo $106.25 (20 kg, Nuevo, Express). Se probaron 8 valores alrededor de esos bordes y del salto de tarifa.

3.3 Tabla de decisión

Idea: cuando el resultado depende de **combinar** condiciones, se arma una tabla con todas las combinaciones. Hay 3 tipos de cliente × 2 tipos de envío = **6 reglas**. Con 10 kg el subtotal siempre es $45.00, así solo cambia lo que se quiere observar.

Regla	Cliente	Envío	Recargo	Descuento	Total
R1	Nuevo	Estándar	$0.00	$0.00	$45.00
R2	Nuevo	Express	$11.25	$0.00	$56.25
R3	Frecuente	Estándar	$0.00	$4.50	$40.50
R4	Frecuente	Express	$11.25	$5.63	$50.62
R5	VIP	Estándar	$0.00	$9.00	$36.00
R6	VIP	Express	$11.25	$11.25	$45.00

3.4 Ejecución manual en la página

Además de automatizarlos, el equipo probó 20 casos a mano en la página (CN-01 a CN-20) y guardó una captura de cada uno. **Los 20 pasaron.** Esta ronda manual encontró el defecto DEF-03, que se explica en la sección 5.

4. Pruebas de caja blanca y cobertura

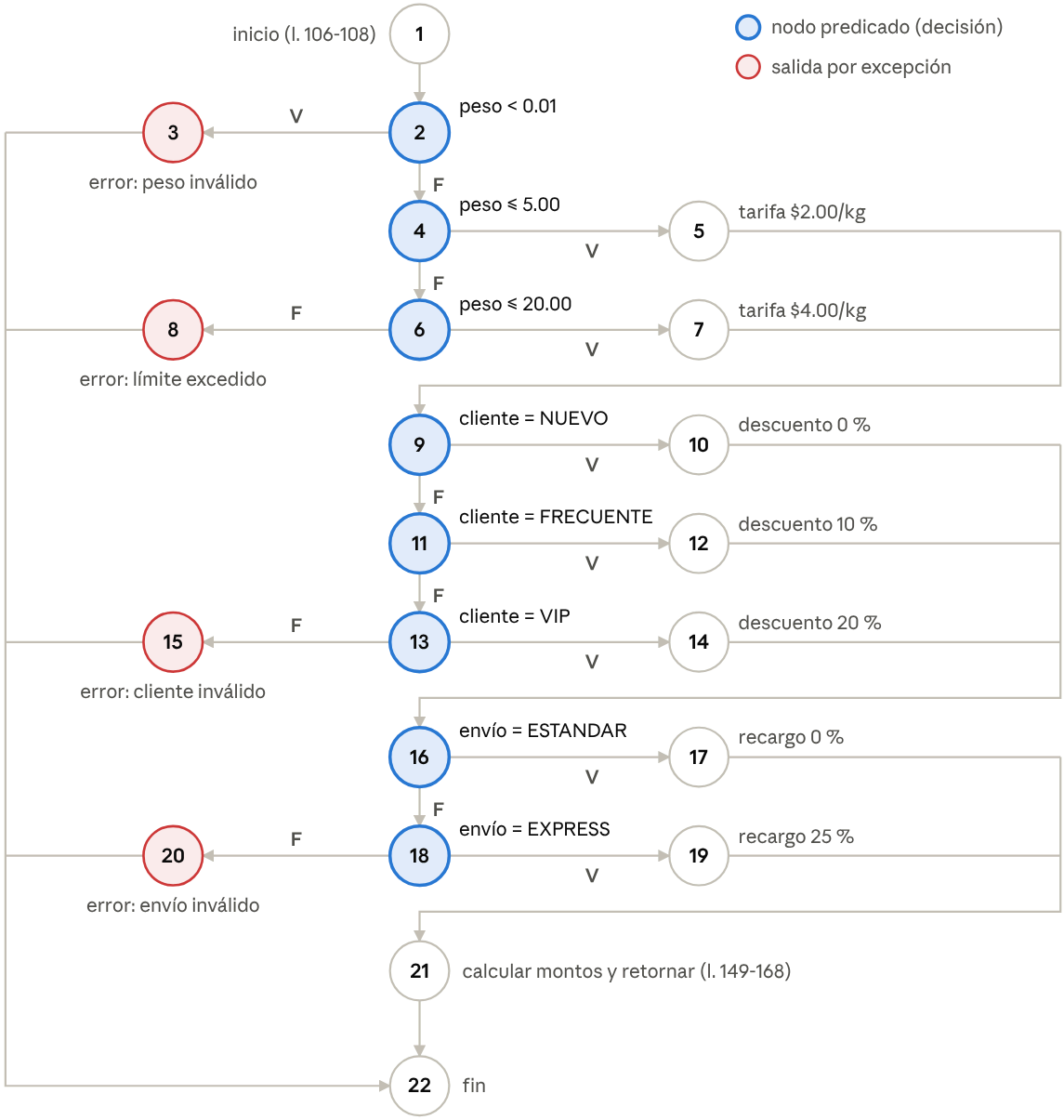
En caja blanca sí se mira el código: el objetivo es **recorrer todos los caminos posibles** de la función `calcular_cotizacion()`. El resultado clave es **V(G) = 9**: hacen falta 9 casos para cubrir todos los caminos independientes, y los 9 están probados.

4.1 El grafo de flujo de control (CFG)

El grafo es un dibujo del código. Cada **nodo** (círculo) es una instrucción o una pregunta, y cada **arista** (flecha) es un paso posible de una a otra. Las preguntas del código (cada `if` o `elif`) son **nodos predicado**: de ellos salen dos flechas, una por "sí" y otra por "no".

Ocho decisiones dan V(G) = 9 caminos independientes

CFG de calcular_cotizacion() · 22 nodos, 29 aristas · V = verdadero, F = falso



La función hace 8 preguntas en orden:

1. ¿Peso menor que 0.01? ¿Peso hasta 5? ¿Peso hasta 20? (3 preguntas sobre el peso)
2. ¿Es Nuevo? ¿Es Frecuente? ¿Es VIP? (3 sobre el cliente)

3. ¿Es Estándar? ¿Es Express? (2 sobre el envío)

4.2 La complejidad ciclomática V(G)

V(G) dice **cuántos caminos independientes** tiene el código, o sea, el número mínimo de pruebas para pasar al menos una vez por cada flecha. Se calcula de tres formas y las tres deben coincidir:

Método	Fórmula	Cálculo	Resultado
Aristas y nodos	E − N + 2	29 − 22 + 2	9
Nodos predicado	P + 1	8 + 1	9
Regiones	regiones cerradas + 1	8 + 1	9

Un V(G) entre 1 y 10 se considera de **riesgo bajo**: el código es simple y fácil de probar.

4.3 Los 9 caminos y sus pruebas

Cada camino agrega al menos una flecha nueva. La prueba `test_caminos_independientes_cfg` los recorre todos.

Camino	Qué recorre	Datos	Resultado
C1	Peso inválido	0 kg, Nuevo, Estándar	Error de peso
C2	Peso sobre el límite	25 kg, Nuevo, Estándar	Error de límite
C3	Tramo 1, Nuevo, Estándar	2.5 kg	$10.00
C4	Tramo 2, Nuevo, Estándar	12 kg	$53.00
C5	Cliente Frecuente	1 kg, Estándar	$6.30
C6	Cliente VIP	1 kg, Estándar	$5.60
C7	Cliente desconocido	1 kg, PREMIUM	Error de cliente
C8	Envío Express	1 kg, Nuevo	$8.75
C9	Envío desconocido	1 kg, URGENTE	Error de envío

4.4 Cobertura de código

La cobertura mide **qué porcentaje del código se ejecutó** durante las pruebas. La herramienta es pytest-cov y mide dos cosas:

- **Líneas:** ¿se ejecutó cada línea al menos una vez? Resultado: 150 de 150.
- **Ramas:** ¿cada `if` se probó por el "sí" y por el "no"? Resultado: 26 de 26.

El proyecto pedía un mínimo de 85 % y se logró **100 %.** Ese mínimo está escrito en `pytest.ini` (`--cov-fail-under=85`), así que si alguien quita pruebas y la cobertura baja, pytest marca la ejecución como fallida.

Ojo, limitación importante: 100 % de cobertura significa que todo el código se ejecutó, no que todo esté bien. Por eso cada prueba también verifica el monto exacto esperado. Y por eso la prueba manual encontró DEF-03 aun con 100 % de cobertura: ese error estaba en la página, que la cobertura de Python no mide.

5. Defectos encontrados y por qué importan

Las pruebas encontraron **3 defectos** y los tres están corregidos. Cada uno lo encontró un tipo de prueba distinto, y esa es la lección principal del proyecto: ningún tipo de prueba alcanza por sí solo.

Defecto	Qué pasaba	Quién lo encontró	Cómo se corrigió
DEF-01 (severidad media)	Si se le pasaba el tipo VIP como objeto `Enum`, el cálculo lo rechazaba. Desde Python 3.11, `str()` de un Enum devuelve "TipoCliente.VIP" y no "VIP"	Una prueba unitaria	`normalizar_texto()` ahora usa `.value` del Enum
DEF-02 (severidad alta)	Abriendo la página con Live Server (puerto 5500), la página no podía hablar con la API (puerto 5000)	Prueba de integración manual	La página usa la dirección completa del servidor y la API envía permisos CORS

Defecto	Qué pasaba	Quién lo encontró	Cómo se corrigió
DEF-03 (severidad media)	Al escribir un dato inválido después de una cotización buena, salía el mensaje de error pero seguía visible el precio anterior. El usuario podía creer que ese precio era el nuevo	Prueba manual de caja negra (CN-10, CN-19 y CN-20)	`app.js` oculta la cotización anterior cada vez que se pulsa Cotizar

Por qué importan:

- **DEF-01** muestra que el código puede romperse al cambiar de versión de Python; las pruebas unitarias lo detectan al instante.
- **DEF-02** muestra que dos partes pueden funcionar bien solas y fallar juntas; para eso están las pruebas de integración. **CORS** es una protección del navegador que bloquea llamadas entre puertos distintos si el servidor no las autoriza.
- **DEF-03** tuvo 100 % de cobertura y aun así existía, porque era un problema de lo que ve el usuario. Solo una persona probando la página lo vio.

Cada corrección tiene una **prueba de regresión**: una prueba automática que falla si el defecto vuelve a aparecer. Por ejemplo, `test_error_de_validacion_oculta_la_cotizacion_anterior` abre un navegador real, cotiza 20 kg, escribe 20.01 y comprueba que el precio anterior ya no se ve.

6. Cómo ejecutar todo

Con Python instalado, todo se hace en cuatro pasos desde la carpeta del proyecto (la que tiene `requirements.txt`). Los comandos son para Windows; en Mac o Linux, el paso 1 usa `source .venv/bin/activate`.

1. **Crear y activar el entorno virtual** (una caja aislada para las librerías del proyecto):

```
python -m venv .venv
.venv\Scripts\activate
```

2. **Instalar las librerías** (Flask, pytest y pytest-cov):

```
pip install -r requirements.txt
```

3. **Correr las pruebas.** Debe decir `82 passed` (o `80 passed, 1 skipped` si no instalaste Playwright, porque el archivo con las 2 pruebas de navegador se salta entero) y `Total coverage: 100.00%`:

```
pytest
```

El reporte de cobertura queda en `reportes/cobertura/index.html`; ábrelo con el navegador.

4. **Levantar la aplicación** y abrir http://127.0.0.1:5000:

```
cd backend
python -m fastexpress.api
```

La terminal debe quedar abierta mientras usas la página. Para detenerla, pulsa Ctrl + C.

Mapa de archivos

Archivo	Para qué sirve
`backend/fastexpress/reglas.py`	Los números de las reglas: tarifas, límites, porcentajes
`backend/fastexpress/errores.py`	Los tipos de error y sus códigos
`backend/fastexpress/cotizador.py`	El cálculo: `calcular_cotizacion()`
`backend/fastexpress/api.py`	La API y el servidor de la página
`frontend/`	La página: `index.html`, `app.js`, `styles.css`
`tests/test_cotizador.py`	Pruebas del cálculo: particiones, límites, tabla de decisión y caminos
`tests/test_api.py`	Pruebas de la API: códigos 200, 400 y 422, y CORS
`tests/test_frontend_e2e.py`	Pruebas en un navegador real (opcionales)
`pytest.ini`	Configuración de pruebas y mínimo de cobertura del 85 %
`docs/`	Informe técnico, casos manuales y las 20 capturas

7. Preguntas probables del docente

Respuestas cortas para practicar en voz alta. Si puedes explicarlas con tus palabras, dominas el proyecto.

¿Cuál es la diferencia entre caja negra y caja blanca? Caja negra prueba lo que el sistema hace según las reglas, sin ver el código. Caja blanca prueba cómo lo hace, recorriendo los caminos del código.

¿Por qué usaron particiones de equivalencia? Porque todos los valores de un grupo se comportan igual; probar uno representa al grupo y ahorra cientos de casos.

¿Por qué probaron 4.99, 5.00 y 5.01? Porque 5.00 es el borde donde cambia la tarifa, y los errores suelen estar en los bordes (por ejemplo, poner `<` en vez de `<=`).

¿Qué es V(G) y cuánto vale aquí? Es la complejidad ciclomática: el número de caminos independientes. Vale 9, calculado con E − N + 2 = 29 − 22 + 2 y con P + 1 = 8 + 1.

¿Qué es un nodo predicado? Un punto del código donde se toma una decisión (un `if`), del que salen dos caminos. Aquí hay 8.

¿Cuánta cobertura lograron y cuánto pedían? 100 % de líneas y de ramas; el mínimo era 85 %.

Si tienen 100 % de cobertura, ¿el programa no tiene errores? No. La cobertura dice que el código se ejecutó, no que el resultado sea correcto. DEF-03 lo demuestra: existía con 100 % de cobertura.

¿Qué diferencia hay entre cobertura de líneas y de ramas? Líneas: cada línea se ejecutó una vez. Ramas: cada `if` se probó por el lado verdadero y por el falso. La de ramas es más exigente.

¿Por qué la tabla de decisión tiene 6 reglas? Porque hay 3 tipos de cliente por 2 tipos de envío: 3 × 2 = 6 combinaciones.

¿Por qué el descuento se calcula después del recargo? Fue un supuesto del equipo porque el enunciado no lo decía. Como ambos son porcentajes, el total es el mismo en cualquier orden.

¿Qué pasa si alguien pide 25 kg? El sistema lo rechaza con el error `LIMITE_EXCEDIDO` y el código HTTP 422.

¿Qué diferencia hay entre el error 400 y el 422? 400: el dato está mal escrito o falta (peso "abc"). 422: el dato es válido pero viola una regla de negocio (25 kg).

¿Qué es una prueba de regresión? Una prueba que se agrega al corregir un defecto para que, si el defecto vuelve, la suite lo detecte.

¿Qué normas siguieron? IEEE 829 e ISO/IEC/IEEE 29119-3 para la estructura del informe, las técnicas de diseño del programa ISTQB, e ISO/IEC 25010 para las características de calidad (adecuación funcional y fiabilidad).

¿Qué rol cumplió cada integrante? Ruben Coronel en Backend, Dilan Ariza en Frontend y Marcos Arce en QA (pruebas y calidad).